

Beating V8 BigInt Multiplication from WebAssembly

Julian Henry

September 2026

Abstract

A WebAssembly (WASM) multiplier beats V8’s native `BigInt` multiply at every operand size we tested from 16 kbit to 33.5 Mbit: $1.2\text{--}4.5\times$ faster in kernel time and up to $2.9\times$ faster including conversion to and from `BigInt`. The winning kernel is a SIMD double-precision FFT with balanced digits, a half-size real-output inverse, and an $O(n)$ residue check with automatic retry. All 1,336 test products matched V8 exactly, up to 1M limbs. A rebuilt in-place Karatsuba scales with the textbook exponent but stays a constant $1.5\times$ behind V8’s own Karatsuba, because V8 multiplies 64-bit digits natively: no Karatsuba in WASM can overtake it; only a lower-exponent algorithm can. We also show that the repository’s previous Karatsuba kernel returned wrong products for most random operands above 32 limbs.

1 Background

We measured V8’s `a*b` (Node 26.5, V8 14.6, Apple M1 Pro). Up to about 2,048 32-bit limbs its time grows with local exponent ≈ 1.6 (Karatsuba); beyond, the exponent is 1.05–1.2 (an FFT-class algorithm). V8 uses 64-bit digits with native $64 \times 64 \rightarrow 128$ -bit multiplies. WASM has none: this V8 rejects the `wide-arithmetic` proposal’s `i64.mul.wide.u`. A WASM kernel therefore performs about $4\times$ more digit products at every size.

Why earlier attempts could not win.

- *Algorithm ceiling.* $O(n^{1.585})$ Karatsuba can at best tie V8’s Karatsuba, and loses to V8’s FFT above $\sim 2K$ limbs.
- *Allocation on the hot path.* The old `karatsuba.wat` bump-allocated and copied every split, sum and partial product; the old schoolbook allocated $\sim 3,000$ blocks (~ 16.7 MB) per 1,024-limb multiply.
- *Benchmark artefacts.* The old harness timed an $O(n^2)$ `BigInt`-to-limbs conversion inside the WASM multiply, and reused the same operands every repetition. V8 hoists such loop-invariant `a*b`, and lowers `(a*b) & 1n` to a 64-bit multiply; naive V8 timings read 3 ns at every size.

2 Methods

All kernels are in `research/src/bigmul.c` (654 lines of C, a 21 KB WASM module with SIMD128). Numbers are little-endian 32-bit limbs; kernels write into caller-provided buffers, so nothing is allocated during a multiply.

Karatsuba, rebuilt. We use the subtractive form

$$a_0b_1 + a_1b_0 = a_0b_0 + a_1b_1 - (a_0 - a_1)(b_0 - b_1), \quad (1)$$

so half-sums never grow a carry limb (the old kernel’s bug class). Recursion uses one $\sim 4n$ -limb scratch buffer. Below 96 limbs a SIMD Comba base case computes each column as a contiguous dot product; `i64x2.extmul` forms two 32×32 products per instruction, with low and high halves in separate 64-bit accumulators (no carry chain).

FFT multiply. Five steps, all SIMD `f64x2` on split `re/im` arrays:

1. **Balanced digits.** Cut operands into b -bit digits in $[-2^{b-1}, 2^{b-1})$, with b the largest value satisfying $2b + \log_2 N \leq 56$ ($b = 15\text{--}20$).
2. **One forward transform.** Pack $z = a + ib$; radix-4 DIF FFT of size N , depth-first until a sub-transform fits in L1 (4,096 points). Each radix-4 group loads one twiddle and derives the rest: $w(j+q) = -i w(j)$, $w_2 = w_1^2$.
3. **Pointwise product in scrambled order.**

$$A_k B_k = \frac{Z_k^2 - \overline{Z_{-k}}^2}{4i}. \quad (2)$$

In bit-reversed order, the mirror of position $p \in [2^m, 2^{m+1})$ is $3 \cdot 2^m - 1 - p$, so no permutation pass is needed.

4. **Half-size inverse.** The product spectrum C is Hermitian; fold it in place ($H = N/2$):

$$Z'_k = (C_k + \overline{C_{H-k}}) + i W^{-k} (C_k - \overline{C_{H-k}}), \quad (3)$$

and $\text{IFFT}_H(Z') = N(c_{2j} + i c_{2j+1})$. Transform work drops from $2N$ to $1.5N$.

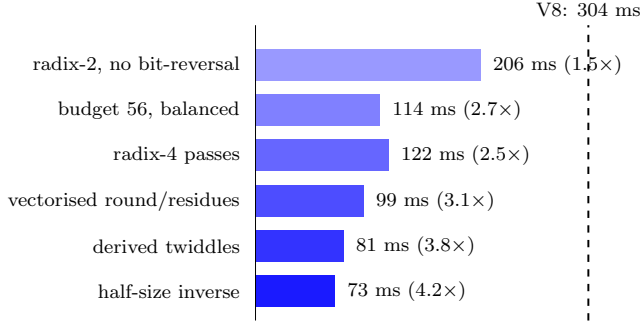


Figure 1: Optimisation ladder at 1M limbs (33.5 Mbit); each step measured after the previous one. The dashed line is V8’s time; ratios are speedups over V8.

5. **Round, carry, verify.** Accept only if the maximum rounding error is below 0.375 and residues mod $2^{32} - 1$ and $2^{31} - 1$ match; otherwise retry with $b - 1$. One wrong coefficient shifts the result by $\pm 2^{bk} \not\equiv 0 \pmod{2^{32} - 1}$, so it is always detected.

`mul_auto` uses Karatsuba below 216 limbs and the FFT above.

3 Experimental setup

Apple M1 Pro (arm64), Node 26.5.0 / V8 14.6.202.34; clang 24.0.0git (`--target=wasm32 -O3 -msimd128`). Sizes 8 to 1,048,576 limbs (256 bit to 33.5 Mbit) at half-octave steps (35 sizes), balanced $n \times n$ products, seeded xorshift128+ operands. Four distinct operand pairs rotate through 16 combinations; every V8 product escapes to a global array; repetitions double until ≥ 60 ms (≥ 400 ms above 64K limbs); five interleaved trials, median reported (min–max spread $< 2\%$ above 8 limbs). Kernel time assumes operands are in WASM memory; end-to-end time adds `BigInt`→`hex`→limbs and back. Correctness: 1,336 exact comparisons against V8 (all sizes 1–80 limbs; $2^p \pm 1$, 2^p , $1.37 \cdot 2^p$ for $p = 7$ –16; 262,144, 777,777 and 1,048,576 limbs; random and all-ones operands; six kernels).

4 Results

The WASM kernel beats V8 at every size from 16 kbit (512 limbs) up: 2.7–4.5× at power-of-two sizes from 64 kbit, 2.1–3.0× just above them (FFT padding). Below ~12 kbit V8 wins by 1.2–1.9× (Fig. 2).

Scaling. Fitted exponents (time $\propto \text{size}^k$): over 4–64 kbit, V8 $k = 1.64$ and WASM Karatsuba $k = 1.65$; over 128 kbit–33 Mbit, V8 $k = 1.18$, WASM Karatsuba $k = 1.59$, WASM FFT $k = 1.08$. Karatsuba runs parallel

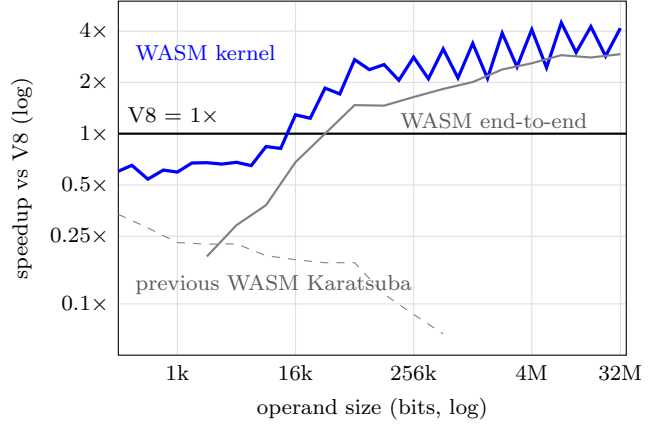


Figure 2: Speedup over V8 BigInt (V8 time / WASM time). The zigzag is FFT padding just above powers of two. WASM first wins at 16 kbit and peaks at 4.5× at 8 Mbit. End-to-end includes the hex round trip.

to V8; only the FFT bends below it (Fig. 3). At 33.5 Mbit Karatsuba is 98× slower than the FFT.

Rounding error. Unsigned digits average 2^{b-1} , so their products add coherently and error grows $\propto N$; balanced digits average zero and add like a random walk. The gap grows with size: 53× at 1K limbs, ~160× at 16K, 1,280× at 1M (Fig. 4). At 1M limbs the policy picks 17 bits, with worst observed error 0.014.

Adversarial inputs. Inputs whose balanced digits are all maximal and of one sign reach error 0.5 on the first attempt. The residue check caught every one; each product was exact after one retry (1K–256K limbs) or two (1M limbs, 458 ms).

End-to-end cost. Converting `BigInts` into WASM memory and back costs about half as much as the multiply itself, so the end-to-end path breaks even with V8 at 32 kbit and reaches 2.9× at 33.5 Mbit (Fig. 5).

Where FFT time goes. The two transforms dominate: the forward transform takes 38–46% and the half-size inverse about 16–21%. The residue check costs 6–9% (Fig. 6).

5 Discussion

Is beating V8 impossible? Not in general, only within the same algorithm class. In the Karatsuba regime both implementations share the exponent, so the 1.47–1.52× constant gap from V8’s 64-bit digits never closes. Above 2,048 limbs V8 switches to its FFT ($k = 1.18$) and WASM Karatsuba falls 23× behind at 1M limbs.

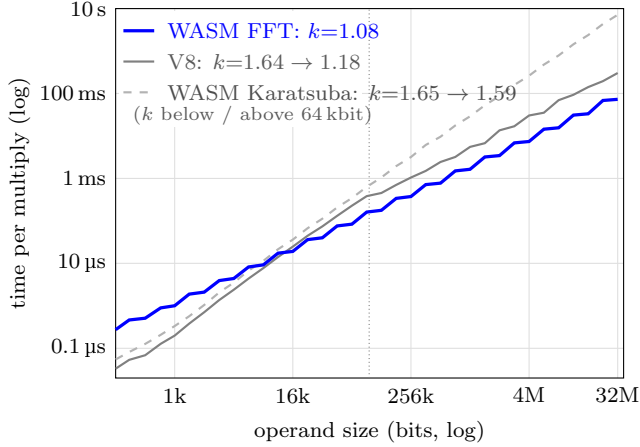


Figure 3: Time per $n \times n$ multiply (log-log) with fitted exponents k (time $\propto \text{size}^k$). WASM Karatsuba stays parallel to V8’s Karatsuba; only the WASM FFT bends below V8’s curve.

Overtaking requires an FFT with better constants than V8’s own; ours scales at $k = 1.08$.

The previous kernel was wrong. `karatsuba.wat` stores $x_{\text{low}} + x_{\text{high}}$ in an $(m+1)$ -limb block but copies only m limbs; a carry then adds stale heap contents. It passes on zeroed memory (how the old tests ran). With a reused heap, random operands failed at 48 limbs in 17/20 trials and at every size ≥ 96 limbs in 20/20; its own recursion reuses memory, so it fails within one call at depth ≥ 2 . One `i32.store` of zero fixes it. Fixed, it is $3.8\times$ slower than the new Karatsuba and $10.6\times$ slower than the FFT at 1,024 limbs.

Limitations. One machine and engine; balanced $n \times n$ only; a padding sawtooth; exactness rests on verification, not proof; adversarial inputs cost retries ($1.5\times$ slower than V8 at 1M limbs); end-to-end pays only from $\sim 1,024$ limbs; FFT memory $\approx 5N$ doubles (~ 170 MB at 1M limbs), capping operands near 20M limbs under `wasm32`.

Future work. `wide-arithmetic` for 64-bit digits; mixed-radix or truncated FFTs; Toom-3 for 128–400 limbs; worker threads; an NTT for proof-level exactness.

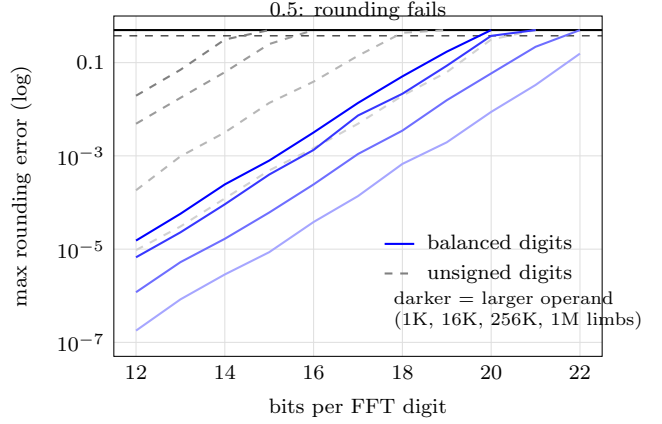


Figure 4: Maximum FFT rounding error vs digit size for random operands of 1K, 16K, 256K and 1M limbs (darker = larger). Balanced digits (solid) stay $50\text{--}1,280\times$ below unsigned digits (dashed); at 1M limbs the policy picks 17 bits (error 0.014).

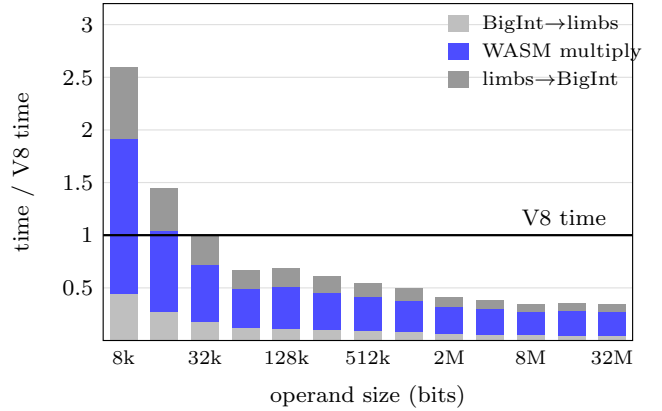


Figure 5: End-to-end time split into conversion in, WASM multiply and conversion out, relative to V8’s `a*b` (bars below the line are faster than V8).

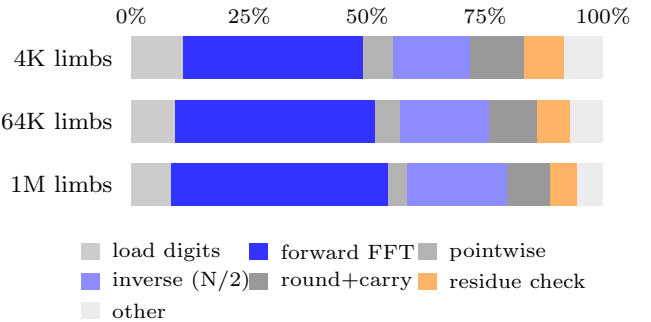


Figure 6: Share of FFT-multiply time per pipeline stage, measured stage by stage. “Other” is the Hermitian fold and call overhead.